POO COM PYTHON

Pedro Henrique Ribeiro Dias

Inatel

INTRODUÇÃO

Diferente de linguagens que sobrecarregam o desenvolvedor com sintaxes rígidas e gerenciamento manual complexo, o Python adota a filosofia de que "legibilidade conta". Na Programação Orientada a Objetos, isso se traduz em um código limpo, intuitivo e extremamente poderoso.

- Gerenciamento Inteligente: Assim como as linguagens modernas, o Python utiliza o Garbage Collector para cuidar da memória, permitindo que você foque na arquitetura do software.

POO COM PYTHON

1. Permite melhor organização, reutilização de código e manutenção facilitada, promovendo sistemas altamente escaláveis e flexíveis.

2. Criamos classes como moldes para objetos que possuem atributos e métodos, ou seja, características e comportamentos

PILARES DA POO

1. Abstração
2. Herança
3. Encapsulamento
4. Polimorfismo

AGREGAÇÃO

Agregação (Relacionamento "Tem"):

- É um tipo de Composição mais fraco.
- Representa uma relação "todo-parte" onde as partes podem existir independentemente do todo.
- Exemplo: Um Carro agrega um Motor. O motor pode ser removido e usado em outro carro.

COMPOSIÇÃO

Composição (Forte): A parte não pode existir sem o todo.

- Exemplo: Um Carro compõe um Chassi. Se o carro for destruído, o Chassi não sobrevive sozinho.

CONSTRUTORES E COLECTIONS

1. Construtores:
   - Método automático que inicializa o objeto.
   - Garante que o objeto nasça em um estado válido.
   - Chamado usando new.
2. Coleções (Listas e Vetores):
   - Usadas para armazenar vários objetos.
   - List<T>: Coleção dinâmica (tamanho variável).
   - Polimorfismo: Permite armazenar objetos de diferentes classes filhas em uma lista do tipo da classe mãe (List<Pai>).

INTERFACES

- Interfaces definem métodos obrigatórios que as classes concretas devem implementar. Servem para garantir consistência e flexibilidade no código.

  Em Python, usamos classes abstratas para representar interfaces. E em Python são criadas utilizando o módulo abc

INTERFACE

1. Criamos uma classe que herda de ABC

2. Definimos métodos com o decorador @abstractmethod

3. As classes concretas implementam esses métodos

4. Exemplo herança de interface (interface de ação):
    a. class Guerreiro(IAcao)
    - Obriga o guerreiro a implementar o método que a interface ação carregar.

POR QUE USÁ-LAS?

- Garantem que diferentes classes, porém que possuem comportamentos semelhantes, compartilhem o mesmo “contrato” para que sejam obrigadas a implementá-lo.
- Facilitam a substituição de implementações.
- Tornam o código mais modular e escalável.
- Permitem o uso de polimorfismo de forma limpa.

INTERFACES E RELAÇÕES

- Nosso exemplo prático implementa um mini- sistema de RPG que demonstra todos os conceitos de POO em Python:
- Interface (IAcao) → define o contrato para métodos atacar() e defender(). Composição → cada Personagem possui uma Arma. Agregação → cada Personagem tem um Inventario que gerencia uma lista de Itens.
- Herança e Polimorfismo→ classesGuerreiroe
- Mago herdam de Personagem e implementam IAcao
- deformas diferentes.
- Este exemplo conecta todos os conceitos vistos, abstração, encapsulamento, herança, polimorfismo, agregação, composição e interfaces

EXERCÍCIOS

OO com Python

Inatel

EXERCÍCIO 1

Cenário: Em Dark Souls, todo morto-vivo carrega almas e Frascos de Estus, que se recarregam nas fogueiras e nunca passam de 10. Os clérigos, além disso, conhecem um milagre próprio.

Tarefa:
1. Crie a classe `MortoVivo`, cujo `__init__` recebe e inicializa `nome` (público), `_almas` (protegido) e `__estus` (privado).
2. Crie os métodos `get_estus()` e `set_estus(quantidade)`. O setter só aceita valores **de 0 a 10**. Fora dessa faixa, ele exibe `"Quantidade de Estus inválida!"` e mantém o valor anterior.
3. Crie o método `mostrar_status()`, que retorna um texto com os três dados (ex.: `"Morto-vivo Petrus | Almas: 2000 | Estus: 5"`).

EXERCÍCIO 1

4. Crie a classe filha `Clerigo`, com o atributo exclusivo `milagre`. Inicialize a parte herdada com `super().__init__(...)` e sobrescreva `mostrar_status()`, reaproveitando o texto do pai com `super().mostrar_status()`.
5. No main, crie um `Clerigo` e exiba o status. Tente definir o Estus como `15` e depois como `10` e exiba o status de novo.
6. Tente imprimir o `__estus` do clérigo diretamente no main, observe o erro e comente a linha.

EXERCÍCIO 2

Cenário: Em Overwatch, todo herói pode usar a sua suprema, mas o efeito depende da função. Os suportes, além disso, conseguem curar a equipe.

Tarefa:

1. Crie a classe base HeroiOverwatch, cujo __init__ recebe codinome e funcao, com o método usar_suprema(), que exibe uma suprema genérica.
2. Crie as classes filhas HeroiTanque e HeroiSuporte, sem __init__ próprio, e sobrescreva usar_suprema() nas duas.
3. Em HeroiSuporte, crie também o método exclusivo curar_equipe().
4. Na main, crie uma lista list[HeroiOverwatch] com um herói de cada classe.
5. Percorra a lista com for chamando usar_suprema() e, usando isinstance, chame curar_equipe() apenas nos suportes.

EXERCÍCIO 3

Cenário: Em Persona 5, um grupo de estudantes chamado Phantom Thieves invade Palácios: mundos criados a partir do coração de pessoas corruptas, cada um conhecido pelo nome do seu dono (ex.: o Palácio de Kamoshida). O líder do grupo, de codinome Joker, luta com a sua Persona, uma criatura que nasce de dentro dele e só existe enquanto ele existir (composição). Ao longo da história, ele recruta aliados: colegas que já existiam antes de entrar no grupo e continuam existindo fora dele (agregação).

Tarefa:

1. Crie a classe Persona, com nome e arcano e o método invocar(), que exibe os dois dados.
2. Crie a classe Aliado, com nome (nome real) e codinome.

EXERCÍCIO 3

3. Crie a classe Lider, que representa o Joker, com:
    - codinome;
    - uma Persona criada dentro do __init__, com nome "Arsène" e arcano "Louco";
    - uma lista _equipe que começa vazia.
4. Em Lider, crie os métodos:
    - recrutar(aliado: Aliado), que adiciona o aliado à _equipe;
    - infiltrar(palacio), que recebe o nome do dono do Palácio, exibe o Palácio invadido, invoca a Persona e lista a equipe.
5. Na main, crie aliados antes do líder. Depois, crie o líder com codinome, recrute os dois e chame infiltrar("").
6. Explique, em um comentário no código, o que é composição e o que é agregação no seu programa.

EXERCÍCIO 4

Cenário: Em Cyberpunk, Night City é uma metrópole futurista controlada por grandes corporações. Para atacá-las, hackers se organizam em células, grupos que coordenam diferentes unidades de rede em um mesmo ataque. Toda unidade segue o mesmo protocolo: precisa saber executar uma invasão. Mas cada tipo de unidade faz isso do seu jeito. O netrunner usa o seu cyberdeck para quebrar o gelo (ICE) de um servidor. Já o drone de vigilância intercepta o sinal da rede.

Tarefa:

1. Interface: crie a classe abstrata IUnidadeDeRede (herdando de ABC), com o método abstrato executar_invasao().
2. Parte composta: crie a classe Cyberdeck, com o atributo modelo.
3. Implementação com composição: crie a classe OperadorNetrunner, que implementa IUnidadeDeRede:
    - o __init__ recebe nome e o modelo do cyberdeck, e cria o Cyberdeck dentro dele;
    - executar_invasao() exibe o nome do netrunner, o modelo do cyberdeck e que ele está quebrando o gelo (ICE) de um servidor.

EXERCÍCIO 4

4. Outra implementação: crie a classe DroneDeVigilancia, que também implementa IUnidadeDeRede:
    c. o __init__ recebe codigo, a identificação do drone;
    c. executar_invasao() exibe o código do drone e que ele está interceptando o sinal da rede.
5. Agregação: crie a classe CelulaHacker:
    c. o __init__ recebe nome e uma lista de membros já criados, anotada como list[IUnidadeDeRede];
    c. iniciar_ataque() exibe o nome da célula e chama executar_invasao() de cada membro (polimorfismo).
6. Na main, crie um netrunner e um drone, junte os dois em uma célula e chame iniciar_ataque().
7. Tente criar um objeto IUnidadeDeRede diretamente e observe o erro. Depois, transforme essa linha em comentário (#) para o programa voltar a rodar.

FIM DA AULA!

zSh3rl0cK/**S01-monitoria**



1 Contributor | 0 Issues | 1 Star | 0 Forks

zSh3rl0cK/S01-monitoria

Contribute to zSh3rl0cK/S01-monitoria development by creating an account on GitHub.

GitHub